Mini-Project: Instructions

data prep $\rightarrow$ ML $\rightarrow$ optimization

DAEN 300 - Fall 2026



Goal and timeline

Goal: use FEMA data to understand, step by step:

1. Who is more likely to get a home **inspection**? (`inspnReturned`)
2. Who is **eligible for repair assistance**? (`repairAssistanceEligible`)
3. How much **damage** FEMA found (`rpfvl`) vs. how much **repair money** people received (`repairAmount`)

Look for differences by **type of house**, **insurance**, **how people registered**, **flooding**, and **disaster**.

Day	Focus	Deliverable
1	Get data, explore, plan	Data dictionary + plan
2	Build models, measure, conclude	Notebook + conclusions

Getting the data

OpenFEMA: IHP Valid Registrations v2

`https://www.fema.gov/openfema-data-page/individuals-and-households-program-valid-registrations-v2`

Pick 2–4 large disasters (many registrations), for example major hurricanes or floods. Write down their `disasterNumber`.

Two ways to get them, choose one:

- **Option A: Download the CSV** from the data page (several GB!), then keep only your disasters while reading it
- **Option B: Use the API** to ask FEMA for only your disasters and columns

Honest note: I have not used this API much myself. Explore it, try things, and share what works!

Option A, download the CSV

The full file is too big to open at once. Read it **in pieces** and keep only the rows and columns you need:

```
import pandas as pd

cols = ["disasterNumber", "residenceType", "ownRent",
        "homeOwnersInsurance", "floodInsurance", "registrationMethod",
        "grossIncome", "householdComposition", "applicantAge",
        "primaryResidence", "selfAssessmentInformation",
        "inspnIssued", "inspnReturned", "rpfvl", "waterLevel",
        "highWaterLocation", "floodDamage", "foundationDamage",
        "roofDamage", "destroyed", "habitabilityRepairsRequired",
        "repairAssistanceEligible", "repairAmount"]
my_disasters = [4332, 4673]                # your choice

parts = []
for chunk in pd.read_csv("IHP.csv",        # your file name
                         usecols=cols, chunksize=500_000):
    parts.append(chunk[chunk["disasterNumber"].isin(my_disasters)])

df = pd.concat(parts, ignore_index=True)
df.to_parquet("ihp.parquet")                 # fast to reload later
```

Option B, use the API

The API returns at most 10,000 rows per request, so ask for "pages" until nothing comes back:

```
import requests, pandas as pd

url = ("https://www.fema.gov/api/open/v2/"
       "IndividualsAndHouseholdsProgramValidRegistrations")
params = {
    "$select": ",".join(cols),          # cols from Option A
    "$filter": "disasterNumber eq 4332 or disasterNumber eq 4673",
    "$orderby": "id",
    "$top": 10000,
    "$skip": 0,
}
pages = []
while True:
    data = requests.get(url, params=params, timeout=120).json()
    rows = data["IndividualsAndHouseholdsProgramValidRegistrations"]
    if not rows: break                  # no more pages
    pages.append(pd.DataFrame(rows))
    params["$skip"] += 10000

df = pd.concat(pages, ignore_index=True)
df.to_parquet("ihp.parquet")
```

Import and first look

```
df = pd.read_parquet("ihp.parquet")

df.shape                                          # rows, columns
df.info()                                         # types and missing values
df.describe()                                     # numeric summaries
df["disasterNumber"].value_counts()               # rows per disaster
df["residenceType"].value_counts()                # types of houses
df.groupby("disasterNumber")["inspnReturned"].mean()   # inspection rate
```

- Do the numbers make sense? Any strange codes or missing values?
- Compare disasters early: they can be very different

Day 1

Get data · Explore · Plan

Day 1 goals

By the end of class you should:

1. Download and import your disasters; record a **baseline** (time, rows, memory)
2. Know every variable you will use: **type, meaning, missing values**
3. Explore **inspection** first, then **eligibility** and **amounts**, with tables and plots
4. Decide which variables to use for each question
5. Submit a **plan**: architecture diagram *and* pseudocode

1.1 Load and measure a baseline

- Load your data **as-is** (default types)
- Record:
  - download / load time (`%time` or `%timeit`)
  - number of rows and columns
  - memory: `df.memory_usage(deep=True).sum()`
- Keep these numbers — they are your "before" on Day 2

Measure	Baseline	After (Day 2)
Load time (s)		
Rows × columns		
Memory (MB)		

1.2 Variables to study

Group	Variables
Household (known at registration)	`residenceType`, `ownRent`, `homeOwnersInsurance`, `floodInsurance`, `registrationMethod`, `grossIncome`, `householdComposition`, `applicantAge`, `primaryResidence`
Self-reported damage	`selfAssessmentInformation`
Inspection results (known after inspection)	`rpfvl`, `waterLevel`, `highWaterLocation`, `floodDamage`, `foundationDamage`, `roofDamage`, `destroyed`, `habitabilityRepairsRequired`
Outcomes	`inspnReturned`, `repairAssistanceEligible`, `repairAmount`
Disaster	`disasterNumber`

- `rpfvl` = damage FEMA **assessed**; `repairAmount` = money **received**
- Found another useful variable? Add it and explain why

1.3 Build a data dictionary

Fill one row per variable you keep:

Field	Type	Meaning	% miss.	Known when?
`ownRent`	object	O / R / unknown		registration
`residenceType`				
`homeOwnersInsurance`				
`registrationMethod`				
`waterLevel`				after inspection
`rpfvl`				after inspection
...				

- Use the OpenFEMA data page for definitions
- "Known when?" decides which question each variable can help answer

1.4 Explore inspection first

Question: who has a better chance of being inspected?

1. What share of registrations were inspected? In each disaster?
2. Inspection rate by `residenceType` and `ownRent`
3. With vs. without `homeOwnersInsurance` or `floodInsurance`
4. By `registrationMethod` (phone, internet, mobile, . . .)
5. By `grossIncome` and `selfAssessmentInformation`

Tools: `groupby(...).mean()`, `pd.crosstab`, bar charts.
Show each result **per disaster**: does the pattern hold in all of them?

1.5 Then explore eligibility and amounts

Use only **inspected** homes:

1. Eligibility rate by `residenceType`, insurance, `floodDamage`, `destroyed`
2. `rpfvl` vs. `waterLevel`: scatter plot. Does more water mean more damage?
3. `rpfvl` (assessed) vs. `repairAmount` (received): how close are they?
4. How many zeros? Histograms of the amounts and of `np.log1p(amount)`
5. Correlation heatmap of the numbers and yes/no variables (`df.corr()`)

Again, compare **across disasters**.

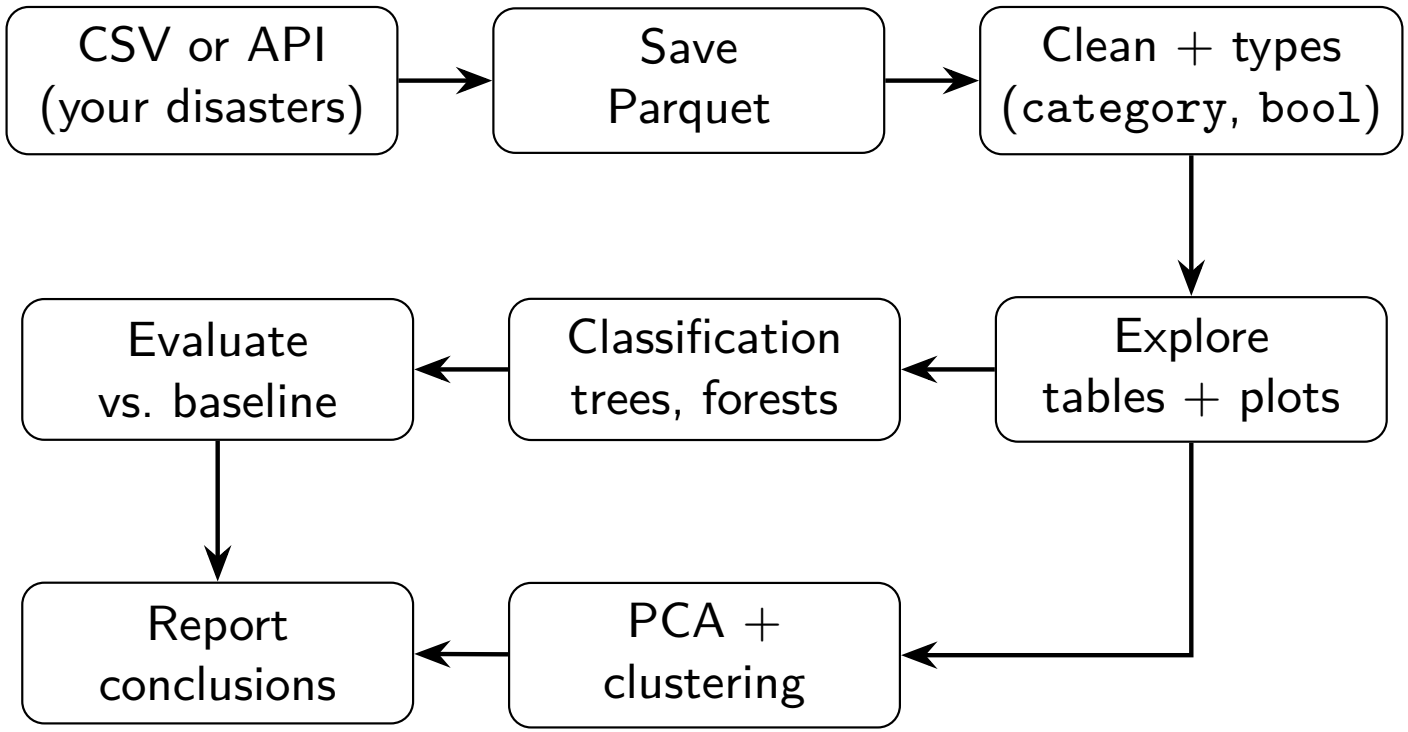
1.6 Which variables for which question?

Rule: use only what is known *before* the outcome.

Question	Rows	Variables
1. Inspected?	all registrations	household + disaster
2. Eligible?	inspected only	household + inspection + disaster
3. Amount received?	eligible only	household + inspection + disaster

- Do **not** use `rpfvl` or `waterLevel` to explain inspection — they come from the inspection itself
- Do **not** use other award amounts as inputs
- **Control by disaster**: include `disasterNumber` as a variable, and check results per disaster
- Write one sentence explaining each variable you keep

1.7 Plan: architecture diagram (example)



Draw *your own*: mark where you will **measure** time or memory.

1.8 Plan: pseudocode (template/example)

```
GET data for my disasters (CSV in chunks, or API pages)
SAVE as parquet;  RECORD time, rows, memory

PREPARE:
    set types (category, bool);  handle missing values
    one-hot encode categories (incl. disasterNumber)

Q1 inspected?    rows = all        X = household + disaster
Q2 eligible?     rows = inspected  X = household + inspection + disaster
Q3 repairAmount? rows = eligible   X = same as Q2

FOR each question:
    split train / test
    fit BASELINE                  -> record score
    fit linear model              -> record score
    (if time) fit logistic        -> record score
    fit tree, random forest       -> record score, top features
    CHECK results per disaster

PCA (2 components) + k-means on inspected homes
DESCRIBE each cluster
REPORT before/after table, plots, conclusions
```

Day 1 deliverable (Fri Nov 13)

Submit on Canvas (notebook (and its PDF export) or just PDF):

- ☐ Which disasters you chose, and why
- ☐ Baseline table: time, rows, memory
- ☐ Data dictionary for every variable you keep
- ☐ Exploration of inspection, then eligibility and amounts (tables/plots with one-line takeaways)
- ☐ Variables chosen for each question
- ☐ Architecture diagram (hand-drawn photo is fine, or draw.io)
- ☐ Pseudocode or algorithm for the full workflow
- ☐ One or two things you want to try to make faster or lighter

Day 2

Build models · Measure · Conclude

Day 2 goals

By the end of class you should:

1. Implement your Day 1 plan (update it if it changed)
2. Try a few speed/memory improvements and **measure** them
3. Build and evaluate models for the three questions, against a **baseline**
4. Use PCA and clustering to find **groups** of similar households
5. Write **conclusions** supported by your numbers

2.1 Optimization: learn by doing

Try it, measure it, keep what helps. No need for big changes.

- Read fewer columns; change types to `category` and `bool` — how much memory did you save?
- Reload from Parquet instead of CSV — how much faster?
- Find one slow step with `%prun`; replace a loop or `.apply()` with a vectorized version or `groupby`
- Random forest slow? Try `n_jobs=-1` or fewer trees

What I tried	Before	After	Worth it?
`category` types	... MB	... MB	
Parquet reload	... s	... s	
...			

2.2 Simple classification and evaluation

Questions 1 and 2 (inspected? eligible?):

- **Baseline**: always predict the most common answer
- **Simple model**: linear regression on the 0/1 outcome (any linear approach from class is fine); predict "yes" if the result is ≥ 0.5
- **If time allows**: try logistic regression and compare
- **Evaluate** on a test set:
  - confusion matrix
  - accuracy, precision, recall, F1
  - cross-validation (`cross_val_score`)
- Are the classes balanced? If not, accuracy can fool you

Question 3 (amount): linear regression on `repairAmount`

ex: `np.log1p(repairAmount)`; report MAE, RMSE, predicting the mean.

2.3 Decision trees and random forests

- **Decision tree**: try depths 2, 4, 8, ...
  - plot train vs. test score by depth — where does it start to overfit?
  - draw a small tree (depth 2–3): can you explain its rules in words?
- **Random forest**: compare with the single tree
- **Feature importance**: which variables matter most?
  - house type? insurance? registration method? water level?
- Use `DecisionTreeRegressor` / `RandomForestRegressor` for `repairAmount`
- Same **evaluation** as 2.2, so you can compare all models in one table

2.4 PCA and clustering

Goal: find groups of similar inspected households.

1. Scale the numeric variables (`StandardScaler`)
2. **PCA** to 2 components; plot them colored by eligibility and by disaster
3. How much variance do 2 components explain?
4. **k-means** with a few values of k; pick one and explain why
5. Describe each cluster: house types, insurance, water level, `rpfvl`, eligibility rate, `repairAmount`

Note: PCA reduces many variables to a few; k-means makes the groups.

2.5 Conclusions

Answer in short paragraphs, each backed by a number, table, or plot:

1. Who is more likely to be **inspected**, and to be **eligible**?
2. How do outcomes differ by **type of house**, **insurance**, **registration method**, and **flooding**?
3. How does `repairAmount` (received) compare with `rpfvl` (assessed)?
4. Do your findings hold **in every disaster**?
5. How much better than the **baseline** is each model?
6. Which optimization helped most?
7. **Limitations** and what you would do with more time

Day 2 final deliverable (Fri Nov 20)

Submit on Canvas: one Jupyter notebook (and its PDF export)

- ☐ Updated plan (diagram + pseudocode), noting any changes
- ☐ Optimization table: what you tried, before/after
- ☐ Model comparison table (baseline, simple, tree, forest)
- ☐ Feature importance and PCA/cluster plots
- ☐ Conclusions
- ☐ Sources and AI use acknowledged

Presentation quality (organization, clarity, tables, graphs, discussion) is part of the grade.